

Sleep & Lifestyle Health Analytics Pipeline

Full Technical Report | Tools, Architecture & Pipeline Structure

Project Type	Real-Time Data Engineering Pipeline
Stack	Apache Kafka PySpark Structured Streaming AWS EC2 S3 Plotly Dash
Versions	Kafka 3.9.2 (KRaft) PySpark 3.5.3 Python 3.x Hadoop-AWS 3.3.4
Environments	Local (Windows + Docker) and Cloud (AWS EC2 t3.micro x2)
Output Storage	Apache Parquet with Snappy compression, written to AWS S3
Dashboard	Plotly Dash — reads from S3 via boto3, refreshes every 45 seconds

1. Project Overview

This pipeline simulates a production-grade real-time health analytics system. Three independent data streams — sleep & lifestyle, personal demographics, and professional context — are continuously generated, ingested through Apache Kafka, cleaned and transformed by PySpark Structured Streaming, stored as columnar Parquet files in AWS S3, and visualised on a live Plotly Dash dashboard.

1.1 Use Case & Motivation

Sleep disorders and lifestyle-driven health degradation affect a significant portion of the working adult population. Traditional health monitoring is reactive — issues are identified only after clinical symptoms appear. This pipeline demonstrates a proactive, real-time approach: continuously ingesting behavioural signals, enriching raw data with derived health metrics, and surfacing patterns that would be invisible in static snapshots.

Problem Signals	Design Goals
- Sleep quality degrades silently before it becomes a clinical issue - Burnout has measurable precursors: overwork hours, meeting load, stress trajectory - Lifestyle factors (caffeine, screen time, exercise) interact non-linearly with sleep - No real-time public dataset exists — pipeline simulation is the industry-standard approach for demonstrating streaming architectures	- Multi-domain streaming: three independent Kafka topics with different schemas and rates - Stateless micro-batch transformations producing clinically meaningful derived columns - Fault-tolerant: checkpointing ensures exactly-once semantics across restarts - Cloud-portable: identical transformation logic runs locally and on AWS EC2 - Cost-aware: entire cloud deployment runs under \$8/month on free-tier infrastructure

1.2 Data Domains

Domain	Kafka Topic	Stream Rate	Key Fields
Sleep & Lifestyle	sleep-lifestyle	Every 2s	sleep_duration_hrs, sleep_quality, stress_level, bmi, steps, caffeine_mg, mood_score
Personal Info	personal-info	Every 5s	age, gender, height_cm, weight_kg, chronic_conditions, sleep_disorder, smoking_status
Profession	profession	Every 8s	job_title, industry, work_hours_per_day, work_stress_score, meetings_per_day, remote_onsite, commute_mins

2. Tools & Technology

2.1 Apache Kafka

Role in Pipeline

Event streaming backbone — decouples producers from consumers, buffers data, and provides ordered, replayable message delivery across three independent topic partitions.

Why Kafka?

- Industry standard for real-time event ingestion — used by LinkedIn, Uber, Airbnb at scale
- Topic-per-domain model isolates schemas: lifestyle, personal, profession streams evolve independently
- Kafka retains messages for 12-24 hours — Spark can replay if processing fails
- Partitioned topics (3 partitions each) allow parallel consumer processing
- KRaft mode eliminates Zookeeper dependency — simpler deployment, fewer processes

Key Configuration

- Mode: KRaft (no Zookeeper) — process.roles=broker,controller on single node
- Kafka 3.9.2 / Scala 2.12 — must match Spark connector Scala version exactly
- Heap: -Xmx512m -Xms256m — tuned down from 1GB default for t3.micro (1GB RAM)
- Dual listeners: INTERNAL://localhost:19092 (local), EXTERNAL://<IP>:9092 (remote Spark)
- advertised.listeners must be public IP — Kafka instructs clients where to reconnect after handshake
- log.retention.hours=12, log.retention.bytes=512MB — conserves EC2 disk space

2.2 Apache PySpark (Structured Streaming)

Role in Pipeline

Consumes all three Kafka topics in parallel micro-batches, applies schema validation, data cleaning, and domain-specific transformations, then writes enriched Parquet files to S3 every 30 seconds.

Why PySpark?

- Only framework that handles streaming and batch transformations in one unified API

Version Matrix

PySpark	3.5.3
Scala build	2.12 — must match Kafka connector
Kafka connector	spark-sql-kafka-0-10_2.12:3.5.3
hadoop-aws	3.3.4 — matches bundled Hadoop

- Structured Streaming is built on Spark SQL — transformations are identical to batch jobs
- Native Kafka source connector via spark-sql-kafka-0-10
- Parquet write with Snappy compression reduces storage 60-70% vs raw JSON
- Checkpointing provides exactly-once delivery semantics across restarts
- Industry-standard in every major data engineering stack — Databricks, EMR, HDInsight

aws-java-sdk	1.12.262 — compatible with hadoop-aws 3.3.4
Java	17 — required by PySpark 3.5.x
Trigger	processingTime 30 seconds
Output mode	append — new records only

2.3 AWS EC2

Role in Pipeline

Two t3.micro instances provide isolated cloud compute: one dedicated to Kafka and data producers, one dedicated to PySpark streaming. Separation prevents OOM conditions given 1GB RAM per instance.

EC2 #1 — Kafka	t3.micro, Ubuntu 24.04 LTS. Runs Kafka 3.9.2 as systemd service + 3 Python producers in tmux. Security group: port 22 (SSH, your IP), 9092 (Kafka, 0.0.0.0/0).
-----------------------	--

EC2 #2 — Spark	t3.micro, Ubuntu 24.04 LTS. Runs PySpark spark-submit reading from EC2 #1. IAM role attached for S3 access. Security group: port 22 only — all outbound traffic is unrestricted by default.
IAM Role	ec2-s3-spark-role with AmazonS3FullAccess. Attached to Spark EC2 — credentials provided automatically via instance metadata endpoint. No hardcoded keys in code.
Cost estimate	EC2 #1 (t2.micro free tier): \$0/month. EC2 #2 (t3.micro): ~\$7.50/month. S3 under 5GB: \$0 free tier. Total: ~\$7.50/month from credits.
Memory constraint	t3.micro = 1GB RAM. Kafka heap capped at 512MB. Spark driver memory set to 700m. Cannot run both on one instance — OOM guaranteed.

2.4 AWS S3

Role in Pipeline

Persistent, durable object storage for Parquet output files and Spark checkpoints. Accessed via the s3a:// protocol using the hadoop-aws connector authenticated through the EC2 IAM role.

Why S3 + Parquet?

- Parquet is columnar — reading only sleep_quality from 1M records scans 1 column, not all
- Snappy compression reduces file size 60-70% vs JSON with fast decompression
- S3 free tier covers 5GB — sufficient for this project indefinitely
- S3 strong consistency (since Nov 2020) means no read-after-write race conditions
- Checkpoints stored on S3 survive Spark EC2 restarts — stream resumes from last offset

S3A vs S3 vs S3N

- s3:// — legacy AWS SDK connector, deprecated, no consistency guarantees
- s3n:// — first Hadoop connector, 5GB file size limit, deprecated
- s3a:// — modern connector, multipart upload, IAM role auth, strong consistency, no size limits
- Bucket structure: output/lifestyle/, output/personal/, output/profession/, checkpoints/
- Auth: InstanceProfileCredentialsProvider reads IAM role from EC2 metadata at 169.254.169.254

2.5 Python Producers & Data Generation

Role in Pipeline

Three independent Python scripts simulate real-world data sources. A persona-based generation model ensures cross-field correlation — a stressed IT worker consistently shows high caffeine, late bedtime, short sleep, and low mood across all three tables.

Personas	7 distinct personas: stressed_tech_worker, healthy_active_adult, night_shift_worker, senior_executive, student_young_adult, retired_senior, finance_professional
User pool	200 synthetic users pre-generated at startup. Each user is assigned a fixed persona — repeat records for the same user_id are internally consistent across all three topics
Kafka key	user_id used as Kafka message key — ensures all records for a user land in the same partition (ordering guarantee per user)
Intervals	lifestyle_producer: 2s (high frequency), personal_producer: 5s, profession_producer: 8s — different rates simulate realistic independent source systems
Library	kafka-python for producer client. Faker for realistic name/location fields. All field ranges defined per persona to prevent cross-field logical contradictions
Why not Faker alone	Faker generates fields independently — a 22-year-old CEO with 0 stress sleeping 11 hours. Personas enforce co-varying ranges making dashboard correlations statistically meaningful

2.6 Plotly Dash Dashboard

Role in Pipeline

Real-time analytics frontend running locally on Windows. Reads Parquet files directly from S3 using boto3 every 45 seconds. No additional server or EC2 required — zero extra cost.

Chart	Data Source	What It Shows
Sleep Duration Histogram	lifestyle	Distribution of sleep hours across population — normal curve centred around 7h
Wellbeing Gauge	lifestyle	Composite index: (sleep_quality + mood - stress)/3 — single glanceable health score
Stress vs Sleep Scatter	lifestyle	Negative correlation between stress and sleep quality, coloured by sleep duration
Sleep Timeline	lifestyle	Sleep duration + stress level over time — dual Y-axis line chart, last 300 records
Sleep Category Donut	lifestyle	insufficient / borderline / optimal / excessive split — clinical categorisation
BMI Distribution	lifestyle	underweight / normal / overweight / obese counts — WHO standard thresholds

Burnout Risk by Industry	profession	Composite burnout index per industry — green/amber/red colour coding
Work Hours vs Sleep	lifestyle + profession joined	Scatter of daily work hours against sleep — coloured by stress level
Remote vs Onsite Pie	profession	Work arrangement distribution — remote workers show different sleep patterns
Lifestyle Factors Correlation	lifestyle	Bar chart of correlation coefficients: exercise/steps positive, alcohol/screen negative
Sleep Disorder Prevalence	personal	none / insomnia / sleep_apnea split by persona type
Age Group Distribution	personal	Population breakdown across 5 age bands — 18-24 through 65+
Activity Level by Persona	lifestyle	Stacked bar: very_active/active/moderate/sedentary split per persona
Hydration Status	lifestyle	dehydrated / adequate / well_hydrated distribution
Overwork % by Industry	profession	Fraction of workforce working >10h/day per industry — IT and Finance highest

3. Pipeline Structure

3.1 Local Version

Environment

Windows 10/11. Kafka runs in Docker Desktop. PySpark runs natively. Output is local Parquet folder. Dashboard reads from local filesystem.

Data Flow

Step	Component	Detail
1	Docker Desktop starts Kafka + Zookeeper on localhost:9092	Kafka 2-container stack, confluentinc 7.5.0, bridge network
2	3 Python producers run in 3 PowerShell windows	lifestyle (2s), personal (5s), profession (8s) — all point to localhost:9092
3	PySpark spark-submit reads all 3 topics	--packages downloads Kafka JAR from Maven, cached after first run
4	Spark micro-batch runs every 30s	Reads new Kafka messages, applies transformations, writes Parquet to local folder
5	Checkpoints written to local folder	Track Kafka offsets — stream resumes correctly after restart
6	Dash dashboard reads local Parquet	Refreshes every 30s — reads all .parquet files in output/lifestyle, personal, profession

Local Project Structure

sleep-pipeline/		
utils/data_generator.py		Persona-based synthetic data generator — 7 personas with correlated field ranges
producers/lifestyle_producer.py		Streams sleep/lifestyle records to sleep-lifestyle topic every 2s
producers/personal_producer.py		Streams demographic records to personal-info topic every 5s

<code>producers/profession_producer.py</code>	Streams work/profession records to profession topic every 8s
<code>spark/stream_processor.py</code>	PySpark Structured Streaming job — reads 3 topics, cleans, transforms, writes Parquet
<code>output/lifestyle/</code>	Snappy-compressed Parquet files — lifestyle stream output
<code>output/personal/</code>	Snappy-compressed Parquet files — personal stream output
<code>output/profession/</code>	Snappy-compressed Parquet files — profession stream output
<code>checkpoints/lifestyle/</code>	Spark checkpoint — Kafka offset tracking for lifestyle stream
<code>checkpoints/personal/</code>	Spark checkpoint — Kafka offset tracking for personal stream
<code>checkpoints/profession/</code>	Spark checkpoint — Kafka offset tracking for profession stream
<code>dashboard.py</code>	Plotly Dash app — reads local Parquet, 15 charts, 45s auto-refresh
<code>docker-compose.yml</code>	Kafka + Zookeeper Docker stack — localhost:9092

3.2 Cloud Version (AWS)

Environment

Two EC2 t3.micro instances on Ubuntu 24.04 LTS. Kafka runs natively as a systemd service. PySpark reads from Kafka EC2 over the public network. Parquet and checkpoints stored in S3. Dashboard runs locally reading from S3 via boto3.

EC2 #1 — Kafka Instance

Instance	t3.micro — Ubuntu 24.04 LTS — free tier eligible
Kafka	3.9.2 KRaft mode, Scala 2.12 — installed to /opt/kafka, runs as kafka system user
Service	systemd unit — auto-starts on boot, restarts on failure
Heap	KAFKA_HEAP_OPTS=-Xmx512m -Xms256m in kafka-server-start.sh

Listeners	INTERNAL://0.0.0.0:19092 (producers, same machine) — EXTERNAL://0.0.0.0:9092 (Spark EC2)
Advertised	INTERNAL://localhost:19092 — EXTERNAL://<PUBLIC-IP>:9092 — must be current public IP
Topics	sleep-lifestyle, personal-info, profession — 3 partitions, replication factor 1
Producers	Run in tmux session — bootstrap_servers=localhost:19092
Security group	Port 22 (SSH, your IP), Port 9092 (Kafka external, 0.0.0.0/0), Port 19092 (internal, 0.0.0.0/0)

EC2 #2 — Spark Instance

Instance	t3.micro — Ubuntu 24.04 LTS — 15GB gp3 storage (Spark needs tmp space)
PySpark	3.5.3 installed via pip3 — spark-submit added to PATH via ~/.bashrc
Extra JARs	hadoop-aws-3.3.4.jar + aws-java-sdk-bundle-1.12.262.jar placed in PySpark jars folder
IAM Role	ec2-s3-spark-role (AmazonS3FullAccess) attached at launch — no credentials in code
S3 auth	InstanceProfileCredentialsProvider — reads from EC2 metadata endpoint 169.254.169.254
Kafka connection	Reads from EXTERNAL listener: kafka.bootstrap.servers=<KAFKA-EC2- PUBLIC-IP>:9092
S3 output	s3a://bucket/output/lifestyle personal profession + s3a://bucket/checkpoints/
Driver memory	--driver-memory 700m — prevents OOM on 1GB RAM instance
Security group	Port 22 only — all outbound unrestricted by default on AWS

Cloud Project Structure (Kafka EC2)

/opt/kafka/	Kafka 3.9.2 installation directory
/opt/kafka/config/kraft/server.properties	KRaft configuration — listeners, log dirs, heap settings
/opt/kafka/kraft-logs/	Kafka internal log storage — topic partition data
/etc/systemd/system/kafka.service	Systemd service unit — auto-start and restart on failure
~/sleep-pipeline/utils/data_generator.py	Persona-based data generator — same file as local version

~/sleep-pipeline/producers/*.py 3 producers — bootstrap_servers=localhost:19092

Cloud Project Structure (Spark EC2)

~/sleep-pipeline/spark/stream_processor.py	Full streaming job — schemas, cleaning, all transformations, S3 write
<pyspark_jars>/hadoop-aws-3.3.4.jar	S3A filesystem implementation for Hadoop/Spark
<pyspark_jars>/aws-java-sdk-bundle-1.12.262.jar	AWS SDK providing S3 API calls to hadoop-aws

4. PySpark Transformations

All transformations are stateless — applied per-record within each micro-batch using `withColumn`. No window aggregations or stateful operations are used, which allows `dropDuplicates` to function correctly in append output mode. Each transformation adds a new derived column to the Parquet output and is directly consumed by the dashboard.

4.1 Lifestyle Stream

Column	Logic	Dashboard Use
sleep_category	< 6h = insufficient, < 7h = borderline, <= 9h = optimal, > 9h = excessive (CDC clinical thresholds)	Sleep Category donut chart
sleep_efficiency_score	(duration/9 * 50) + (quality/10 * 50) — blends objective hours with subjective rating into 0-100 score	KPI card and gauge context
stress_tier	1-3 = low, 4-6 = moderate, 7-10 = high — buckets raw score for grouping	Stress axis labels, scatter colouring
bmi_category	< 18.5 underweight, < 25 normal, < 30 overweight, >= 30 obese — WHO standard	BMI category bar chart
activity_level	steps >= 10000 AND exercise >= 30 = very_active; OR conditions for active/moderate; else sedentary	Activity Level by Persona stacked bar
wellbeing_index	(sleep_quality + mood_score + (11 - stress_level)) / 3 — composite 1-10 wellness score	Wellbeing gauge (primary KPI)
high_stimulant_flag	caffeine >= 200mg AND screen_before_bed >= 60 mins — compound risk flag	Segment filter for sleep analysis
hydration_status	< 1.5L dehydrated, < 2.5L adequate, >= 2.5L well_hydrated	Hydration status donut chart
alcohol_risk_flag	>= 4 units in session — NHS single-session binge threshold	Risk segmentation
bedtime_shift	20-22h = early, 23h = normal, 0-1h = late, else very_late — chronotype classification	Persona sleep pattern analysis

4.2 Profession Stream

Column	Logic	Dashboard Use
--------	-------	---------------

burnout_risk_index	$(\text{hours}/16 * 40) + (\text{stress}/10 * 40) + ((10 - \text{wlb})/10 * 20)$ — weighted 0-100 composite score	Burnout Risk by Industry bar chart
burnout_risk_label	≥ 70 critical, ≥ 50 high, ≥ 30 moderate, else low — from index	Bar chart colour coding (red/amber/green)
overwork_flag	$\text{work_hours_per_day} > 10$ — boolean flag	Overwork % by Industry chart
work_intensity_score	$(\text{hours}/16 * 50) + (\text{meetings}/10 * 25) + (\text{screen}/12 * 25)$ — separates deep work from meeting load	Work intensity segmentation
stress_tier	Same bucketing as lifestyle stream — low/moderate/high from work_stress_score	Industry stress comparison
meeting_load	0-2 = light, 3-5 = moderate, > 5 = heavy — meeting frequency classification	Meeting load breakdown
commute_burden	0 min = none, ≤ 30 = low, ≤ 60 = moderate, > 60 = high	Remote vs commute correlation
income_tier	Ordinal 1-6 mapping from income_bracket string — enables numeric sorting and ranking	Income vs sleep correlation
screen_overuse_flag	$\text{screen_time_hrs} > 8$ — boolean, workplace screen exposure risk	Risk flag aggregation

4.3 Personal Stream

Column	Logic	Dashboard Use
bmi_derived	$\text{weight_kg} / (\text{height_cm}/100)^2$ — recalculated from source fields to cross-validate producer data	Personal stream BMI analysis
bmi_category	Same WHO thresholds as lifestyle stream — applied to derived BMI	BMI category distribution
age_group	18-24, 25-34, 35-49, 50-64, 65+ — standard demographic cohort bucketing	Age Group bar chart
life_stage	< 30 young_adult, < 50 middle_adult, ≥ 50 senior — broader grouping for trend analysis	Life stage cohort segmentation
health_risk_score	Sum of 3 binary flags: chronic_condition (+1), smoker (+1), on_medications (+1) — additive risk 0-3	Population health risk profiling

has_sleep_disorder	sleep_disorder != none — boolean covering insomnia and sleep_apnea	Sleep Disorder Prevalence donut
is_smoker	smoking_status in [occasional, regular] — boolean for lifestyle risk analysis	Risk factor segmentation

5. Data Cleaning Strategy

Cleaning is applied before transformations in each micro-batch. All filters use `.filter()` which is a narrow transformation — no shuffle, no stage boundary. `dropDuplicates` on `[user_id, timestamp]` handles Kafka at-least-once delivery without removing legitimate repeat events from the same user at different times.

Lifestyle Cleaning Rules		Profession + Personal Rules	
user_id	isNotNull — drops records	work_hours_per_day	between(0.0, 20.0)
sleep_duration_hrs	between(2.0, 14.0) — phy	work_stress_score	between(1, 10)
sleep_quality	between(1, 10) — valid sc	work_life_balance	between(1, 10)
heart_rate	between(30, 200) — clinic	age	between(1, 110)
bmi	between(10.0, 70.0) — ex	height_cm	between(50, 250)
steps	>= 0 — no negative step c	weight_kg	between(10.0, 300.0)
stress_level	between(1, 10) — valid sc	String fields	lower(trim()) — normalise
mood_score	between(1, 10) — valid sc	Dedup key	[user_id, timestamp]
Dedup key	[user_id, timestamp]		

6. Local vs Cloud — Side-by-Side Comparison

Component	Local Version	Cloud Version
Kafka	Docker Desktop — confluentinc/cp-kafka:7.5.0	Native binary 3.9.2 KRaft — systemd service on EC2 #1
Zookeeper	Required — runs as separate Docker container	Not needed — KRaft embeds metadata quorum in Kafka
Kafka connect	localhost:9092 — all components on same machine	INTERNAL:19092 (local) + EXTERNAL:9092 (Spark EC2)
PySpark	Runs on Windows — requires winutils.exe for file writes	Runs on Ubuntu EC2 #2 — no winutils needed
Output storage	Local folder — C:/sleep-pipeline/output/	AWS S3 — s3a://bucket/output/ via hadoop-aws

Checkpoints	Local folder — C:/sleep-pipeline/checkpoints/	AWS S3 — s3a://bucket/checkpoints/ — survives restarts
Auth	No auth needed — all local	IAM Role on EC2 — InstanceProfileCredentialsProvider
Dashboard	Reads local Parquet files directly	Reads S3 Parquet via boto3 on local Windows machine
Producers	3 PowerShell windows on Windows	tmux session on Kafka EC2 — persist after SSH disconnect
Cost	\$0 — runs entirely on local hardware	~\$7.50/month — one t3.micro beyond free tier
Fault tolerance	Manual restart required if Spark crashes	S3 checkpoints allow automatic resume from last offset
Resume	interview value: demonstrates the concept	Resume value: production-realistic AWS deployment

7. End-to-End Startup Sequence

Local Startup Order

1. docker-compose up -d (starts Kafka + Zookeeper)
2. Verify: docker ps — both containers Up
3. PowerShell 1: python producers/lifestyle_producer.py
4. PowerShell 2: python producers/personal_producer.py
5. PowerShell 3: python producers/profession_producer.py
6. PowerShell 4: spark-submit --packages ... stream_processor.py
7. Wait 30s — verify Parquet files appear in output/
8. PowerShell 5: python dashboard.py
9. Open browser: http://localhost:8050

Cloud Startup Order

10. Start both EC2 instances in AWS Console
11. SSH into Kafka EC2 — verify: sudo systemctl status kafka
12. On Kafka EC2: tmux new -s producers
13. Run all 3 producers in tmux panes (Ctrl+b to split)
14. Verify: kafka-topics.sh --list --bootstrap-server localhost:19092
15. SSH into Spark EC2 — verify: aws s3 ls (confirms IAM role)
16. Run: spark-submit --packages ... --driver-memory 700m stream_processor.py
17. Wait 30s — verify: aws s3 ls s3://bucket/output/ --recursive
18. On Windows: python dashboard.py — open http://localhost:8050